

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1717

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I **MONIKA M (192521435)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

ANSWERS TO SCENARIO-BASED QUESTIONS

Question 1 — AI-Powered Drone Delivery in a Flood-Affected Region

i. How would Greedy Best-First Search behave here?

Greedy Best-First Search only looks at the heuristic value $h(n)$ — in this case the straight-line distance to the destination — and always jumps to whichever waypoint looks closest on the map. It doesn't care what it actually cost to get there. So the drone would keep chasing the point that "looks" nearest, even if that direction later turns out to be blocked or dangerous.

- Strength: it's quick and doesn't need much memory, which matters when a decision has to be made in seconds.
- Strength: if the heuristic happens to be reliable and nothing changes on the ground, it can reach the goal fast.
- Weakness: it isn't guaranteed to find the best (or even a safe) route — a path that looks short in straight-line terms can hide a flooded or blocked road.
- Weakness: because it ignores cost so far, it can walk straight into a dead end and then have to backtrack, wasting precious time in an emergency.

ii. How would A* handle the same situation?

A* is a bit smarter about it — it scores each node using $f(n) = g(n) + h(n)$, so it weighs the actual distance/cost already travelled (g) together with the estimated distance still remaining (h). That means the drone isn't just chasing "closest looking" points; it's also keeping track of how much real effort (fuel, time, bad terrain) it has spent so far. As long as the heuristic never overestimates the true remaining cost (i.e. it's admissible), A* will end up finding the actual cheapest/safest route, not just a route that looked good early on.

iii. What happens if the heuristic isn't very accurate?

For Greedy Search this is a big deal — the heuristic is basically the only thing steering the search, so a bad estimate can send the drone off in completely the wrong direction. A* is more forgiving: a weaker heuristic mostly just means it explores more nodes before finding the answer (in the worst case, with $h = 0$, it behaves like plain Uniform-Cost Search), but it still finds the optimal route as long as the heuristic doesn't overestimate. If the heuristic does overestimate, though, both algorithms can be led astray, and A* loses its optimality guarantee too.

iv. What effect do dynamic changes (blocked roads) have?

Neither algorithm was really built with a constantly-changing map in mind — both assume the graph they're searching stays put. So the moment a road gets blocked, both need to re-plan. Greedy has it worse here because it doesn't keep any cost history, so it's almost starting over. A* keeps its g -values, which helps, but for a genuinely dynamic setting like this, real systems usually switch to something like D* Lite or Anytime Repairing A*, which are built to reuse earlier search work instead of recalculating everything from zero every time a road closes.

v. Which algorithm would I recommend, and why?

I'd go with A* (ideally a replanning version like D* Lite) over plain Greedy Search. This is a life-critical delivery, so getting the route wrong or stuck isn't really acceptable just to save a few seconds. A* still uses the heuristic to stay efficient, but it doesn't gamble on the route the way Greedy does. If the drone genuinely can't afford the extra computation time, a weighted A* (heuristic slightly inflated) is a reasonable middle ground —

close to optimal but noticeably faster.

Aspect	Greedy Best-First Search	A* Search
Scoring function	$f(n) = h(n)$	$f(n) = g(n) + h(n)$
Optimal route guaranteed?	No	Yes (with admissible heuristic)
Speed	Faster	Slightly slower
Memory needed	Low	Higher
Coping with a blocked road	Struggles, easily misled	Better, but still needs replanning
Fit for this scenario	Risky	Better choice

Question 2 — Smart City Traffic Signal Optimization

Formulating the problem, and why plain search doesn't work here

This is really an optimization problem rather than a "find a path to a goal" problem. The state is the full set of signal timings across every intersection in the city, and we're trying to minimise a combination of waiting time, fuel use, and congestion. The catch is that the number of possible timing combinations explodes as more intersections are added, there's no single fixed "goal state" to search for, and traffic keeps changing while we're computing. Systematic search methods like BFS, DFS, or even A* expect a reasonably sized space with a clear goal — they just aren't practical here. Local search and metaheuristic methods, which improve a candidate solution step by step, fit this problem far better.

i. How would Hill Climbing work, and where does it fall short?

Hill Climbing starts with some timing configuration and keeps moving to a neighbouring configuration that's slightly better, stopping once no nearby change helps. It's simple and cheap to run, which is appealing for a quick fix.

- It tends to get stuck at a local maximum — a setup that looks good locally but isn't the best possible city-wide.
- There's no backtracking, so a bad starting point can trap it early.
- It only looks one step ahead, so it can't really cope with a search space this large and this dynamic.

ii. Local maxima, plateaus and ridges — what do these actually mean here?

A local maximum happens when one junction's timing has been tuned as well as it can be on its own, but the wider traffic flow could still improve if a few junctions were changed together — the algorithm just stops because no single small change helps anymore. A plateau is when tweaking the timing by a second or two barely changes anything measurable, so the search just wanders without making progress until a bigger threshold is crossed. A ridge shows up when real improvement needs several connected intersections to change at the same time — say, synchronising a whole street — but since Hill Climbing only changes one variable at a time, each individual step looks like it's making things worse, even though the combined change would actually help.

iii. How do Simulated Annealing or Genetic Algorithms get past this?

Simulated Annealing occasionally accepts a worse move on purpose — how often depends on a "temperature" that cools down over time — which gives it a chance to climb out of a local maximum or cross a plateau early on, while still settling into a strong solution eventually. Genetic Algorithms take a different route: they keep a whole population of candidate timing plans at once and evolve them through selection, crossover and mutation. Because many solutions are explored in parallel, it's much harder for the whole

population to get trapped in one bad local optimum, and this also makes GAs naturally good at juggling multiple goals at once (waiting time, fuel, congestion) and re-adapting as traffic patterns shift through the day.

iv. Which approach would scale best, and why?

For a city with hundreds of intersections, I'd lean towards a Genetic Algorithm, possibly combined with a faster local search for fine-tuning. It parallelises well, handles multiple competing objectives naturally, and can be re-run periodically as traffic conditions change through the day. Simulated Annealing is a solid lighter-weight option for optimising a smaller area quickly. In practice, a hybrid makes the most sense — GA for the big, periodic re-optimisation, and quick local search for the small adjustments in between.

Question 3 — Autonomous Mars Rover Navigation

i. Why does this need an online search agent instead of offline search?

Offline search only works if we already know the whole map before we start, so the plan can be worked out completely in advance. That's not possible on Mars — the rover doesn't know what's around the next rock until it actually senses it. An online search agent instead acts, observes what happened, updates its picture of the world, and re-plans from there. Given how little is known about the terrain up front, this interleaving of action and sensing is really the only workable approach.

ii. What challenges come from partial observability and a changing environment?

Because the rover can only sense its immediate surroundings, it's constantly working from an incomplete and somewhat uncertain picture of the world — which means there's always a risk of running into a crater or rock it simply couldn't see yet. That pushes it towards more cautious behaviour and keeping track of what it does and doesn't know about nearby terrain. On top of that, the environment itself isn't fixed — dust storms or small rockslides can change the terrain after the rover has already mapped it — so it can't just trust old readings forever; it needs to re-check its surroundings from time to time and re-plan when something's changed.

iii. How does the rover build and update its internal model?

As it moves, the rover keeps updating something like a grid-based map, marking each cell as safe, hazardous, or still unknown, based on what its sensors pick up. Every new observation refines that map and updates the cost/heuristic estimate for nearby cells whose true difficulty turned out different from what was assumed. Algorithms such as Online-DFS or Learning Real-Time A* (LRTA*) work exactly this way — they use the current best estimate to decide the next move, then correct that estimate once the real cost is observed.

iv. How does this compare with uninformed and informed offline search?

Uninformed search like BFS or DFS needs the whole state space known up front and has no real sense of direction toward the goal — not practical for unmapped terrain. Informed offline search like A* also needs a complete map to compute a heuristic-guided path in advance, so it hits the same wall. Online search agents get around this by combining acting with sensing — using local heuristic estimates to pick the next move and learning as they go — but the trade-off is that they generally can't promise a globally optimal path the way offline A* can, since decisions are made with incomplete information.

v. What's the best approach here, and why?

An online, informed strategy like LRTA* fits best — it keeps updating its model of the terrain as it senses more of it, which is exactly what's needed under partial observability and an uncertain, shifting environment. Given how much is riding on the rover's safety, I'd lean towards a more cautious version — one that only commits to moves it's fairly confident are safe and builds in some margin of error — rather than a variant

tuned purely for speed.

Question 4 — University Exam Timetable Generation (CSP)

Setting this up as a CSP — variables, domains and constraints

Each course/exam that needs to be scheduled is a variable ($E_1, E_2, \dots E_n$). The domain of each variable is the set of valid (timeslot, hall) pairs it could be assigned from the available slots and rooms. The constraints tying everything together are:

- No overlap: two exams that share even one student can't be given the same timeslot.
- Hall capacity: the number of exams running in the same slot can't exceed the number of halls available, and each hall must be big enough for the students assigned to it.
- Ordering: certain subjects have to be scheduled before others (e.g. a prerequisite exam).
- Faculty availability: whoever is invigilating an exam has to actually be free at that time.
- Special accommodations: students who need extra time or a separate room have to be placed in slots/halls that support that.

i. How would backtracking search go about solving this?

Backtracking search picks exams one at a time — usually starting with whichever exam is most constrained (fewest valid options left) — and assigns it a (timeslot, hall) pair. After each assignment it checks that nothing conflicts with what's already been scheduled. If a conflict shows up, it backs up to the last exam it assigned and tries a different slot/hall instead. This carries on until every exam has a consistent slot, or it runs out of options to try.

ii. How does constraint propagation (forward checking) help?

Forward checking removes options from other exams' domains the moment they'd conflict with the exam just assigned — so if Exam A shares students with Exam B and Exam A just got a timeslot, that same timeslot is immediately taken off Exam B's list of possibilities. This catches dead ends much earlier than plain backtracking would, since we're not waiting to discover the conflict several assignments later. It cuts down the amount of searching needed quite a bit, and stronger techniques like arc consistency (AC-3) can prune even further before any assignment is even tried.

iii. What real-world challenges come up, and what's the best strategy for scaling?

As the number of courses, students, halls and faculty grows, the constraint graph gets much denser and plain backtracking starts to slow down badly. In practice, it helps to combine backtracking with forward checking and arc consistency, pick the next exam to schedule using heuristics like Minimum Remaining Values or the Degree heuristic, and choose which slot to try first using Least Constraining Value. For a really large university, local search methods such as Min-Conflicts — starting from a full but possibly inconsistent timetable and repeatedly fixing conflicts — often scale better than exhaustive backtracking. So overall, a mix works best: CSP with backtracking and forward checking to get a feasible timetable, then local search to refine it and keep things scalable as the university grows.

Question 5 — Strategic Game AI — Minimax & Alpha-Beta Pruning

i. How does Minimax model this problem?

The game gets represented as a tree, where each node is a game state and each branch is a possible move. The AI (maximiser) tries to pick moves that push the score as high as possible, while the opponent (minimiser) is assumed to always pick moves that push it back down. Minimax works backward from the leaves, computing

the best value each player can guarantee at every node, and that value bubbles up to the root so the AI knows which move gives it the best outcome assuming the opponent also plays well.

ii. What makes large game trees computationally difficult?

The tree size grows exponentially — roughly branching-factor raised to the depth of the search. In a real-time strategy game, there can be a huge number of possible actions at every turn across many units, and the game goes on for a long time, so a full-depth Minimax search simply isn't realistic within a strict time budget. Both the time to search and the memory to hold the tree become a problem fast.

iii. How does Alpha-Beta pruning help?

Alpha-Beta pruning keeps track of two running bounds while going through the tree — alpha, the best score the maximiser can currently lock in, and beta, the best the minimiser can currently lock in. The moment a branch shows it can't possibly change the outcome (its value falls outside that alpha-beta window), the rest of that branch gets skipped entirely. This doesn't change the final answer Minimax would give — it just avoids wasting time exploring branches that couldn't matter. With good move ordering, this can roughly cut the effective branching factor down to its square root, letting the AI search much deeper in the same amount of time.

iv. What would a good evaluation function look like?

Since it's usually not possible to search all the way to the end of the game, the search gets cut off at some depth and a heuristic evaluation function estimates how good that position is. It needs to be quick to compute, since it gets called at a huge number of nodes, and it needs to actually track how likely that position is to lead to a win. Some factors worth including:

- Material/unit advantage — how many units or how much strength each side has relative to the other.
- Positional control — key territory or map areas held.
- Resources — what's available to sustain future moves.
- Unit health — the overall condition of the AI's forces.
- Mobility — how many good moves are still open.
- Threat exposure — how vulnerable the current position is to attack.

v. How should optimality be balanced against real-time constraints?

In practice, Alpha-Beta pruning combined with good move ordering (killer moves, history heuristics) gets the most out of the time available. Iterative deepening is worth adding too, since it means there's always a usable "best move so far" even if the search gets cut off early, and transposition tables help avoid re-evaluating positions the search has already seen. For games with an especially large branching factor, Monte Carlo Tree Search (MCTS) is worth considering either instead of, or alongside, Alpha-Beta. Put together, these give the AI strong decisions without blowing past the time limit.